

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	Assessment:	Constraint Based Problem Solving
Weightage:	40%	Total Marks:	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	ABDUL WAHID. H	Reg. No.:	192321079
Section / Batch:	B.TECH IT	Date:	24/08/26

Instructions to Students

- Identify the relevant concepts of I/O interfaces, interrupts, DMA, bus arbitration, and high-speed communication protocols.
- Analyze the I/O requirements considering CPU utilization, latency, throughput, bandwidth, and device priorities.
- Apply suitable I/O techniques such as DMA, vectored interrupts, memory-mapped I/O, nested interrupts, and bus arbitration.
- Compute performance measures such as data transfer rate, interrupt latency, CPU utilization, throughput, and transfer time.
- Interpret the results by evaluating CPU efficiency, communication performance, interrupt response, and suitability of the proposed I/O architecture.

QUESTIONS

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency $< 10 \mu s$
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct

*I **ABDUL WAHID. H / 192321079** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: ABDUL WAHID. H

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Q. No	Problem Understanding (25)	Constraint Analysis (25)	Solution Development (25)	Application of Concepts (15)	Justification (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/25	/ 25	/ 15	/ 10	/ 100

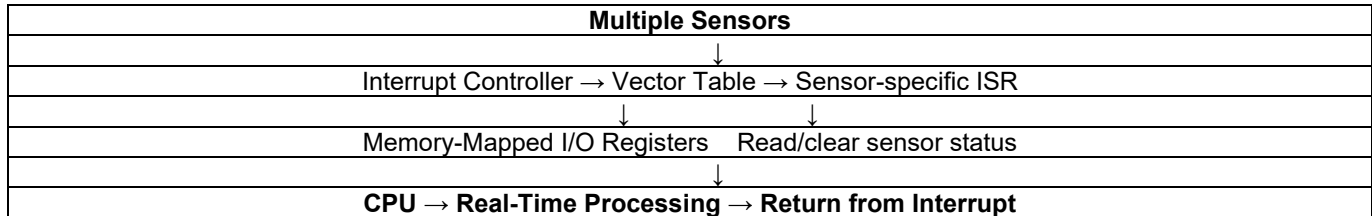
Course Coordinator

Faculty Incharge

Question 1 – Real-Time Embedded I/O Communication Mechanism

Given constraints: CPU utilization < 15%, multiple sensors generate interrupts every 1 ms, vectored interrupt handling, and a choice between memory-mapped and I/O-mapped I/O for low latency.

Proposed Architecture



Design

- Use memory-mapped I/O (MMIO) for sensor control/status registers. The CPU accesses device registers using normal load/store instructions, giving a simple and fast interface suitable for a low-latency embedded system.
- Use vectored interrupts so each sensor/interrupt source maps directly to its ISR address. The CPU therefore avoids scanning all devices to determine the source.
- The interrupt controller should assign priorities to sensors. Time-critical sensors receive higher priority, while less-critical sensors receive lower priority.
- Each ISR should be very short: acknowledge the interrupt, read the required register/data, store the event in a buffer, and defer heavy processing to the main task/background routine.
- Use buffering so short bursts of sensor events do not force long CPU processing inside the ISR.

CPU Utilization Constraint

If there are N sensors and each sensor generates 1000 interrupts/second, the total interrupt rate is:

$$\text{Interrupt rate} = N \times 1000 \text{ interrupts/second}$$

If the average ISR execution time is T seconds, interrupt CPU utilization is:

$$\text{CPU utilization} = N \times 1000 \times T \times 100\%$$

To keep interrupt-related CPU utilization below 15%:

$$N \times 1000 \times T < 0.15$$

Therefore, the maximum average ISR time is $150 \mu\text{s}/N$ per interrupt. For example, if there are 5 sensors, the ISR budget is less than $30 \mu\text{s}$ per interrupt. The exact numerical utilization cannot be calculated without the number of sensors and ISR execution time because those values are not specified in the question.

Why MMIO is Selected

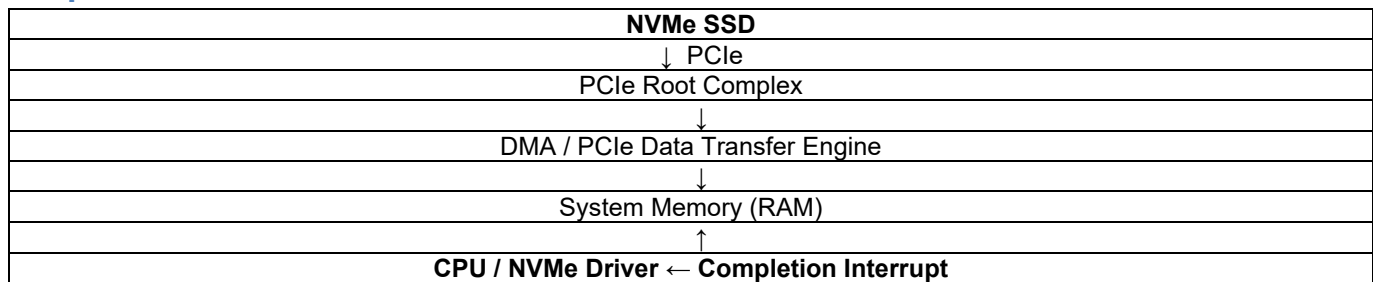
- MMIO gives direct register access using normal CPU memory instructions.
- It avoids a separate I/O address-space instruction mechanism and is convenient for embedded processors.
- Combined with vectored interrupts and short ISRs, it supports low interrupt latency.

Result: The architecture satisfies the stated real-time requirements while keeping CPU utilization below the 15% target when the ISR budget is maintained.

Question 2 – High-Speed NVMe Storage over PCIe

Given constraints: storage throughput > 3 GB/s, NVMe over PCIe, CPU must not handle bulk data transfer, and DMA must be integrated with interrupt notification.

Proposed Architecture



Design

- Use an NVMe SSD connected through PCIe. NVMe is designed for high-speed, queue-based storage communication.
- Use a PCIe link with sufficient lane bandwidth. PCIe 4.0 ×4 provides about 7.88 GB/s of usable one-direction bandwidth under typical encoding overhead, which is comfortably above the required 3 GB/s target.
- Use DMA so storage data moves directly between the NVMe device and system memory without the CPU copying every data block.
- The CPU only prepares NVMe submission queues, manages descriptors/commands, and handles completion notifications.
- After a transfer completes, the NVMe controller generates an interrupt. The driver processes the completion and submits the next operation.
- Use multiple NVMe queues where supported to maintain high parallelism and throughput.

Throughput Check

Required throughput > 3 GB/s

A PCIe 4.0 ×4 connection has substantially more than 3 GB/s of one-direction link capacity, so the link is not the limiting factor for the stated requirement. Actual storage throughput depends on the SSD, controller, queue depth, workload, and system configuration.

Why DMA is Necessary

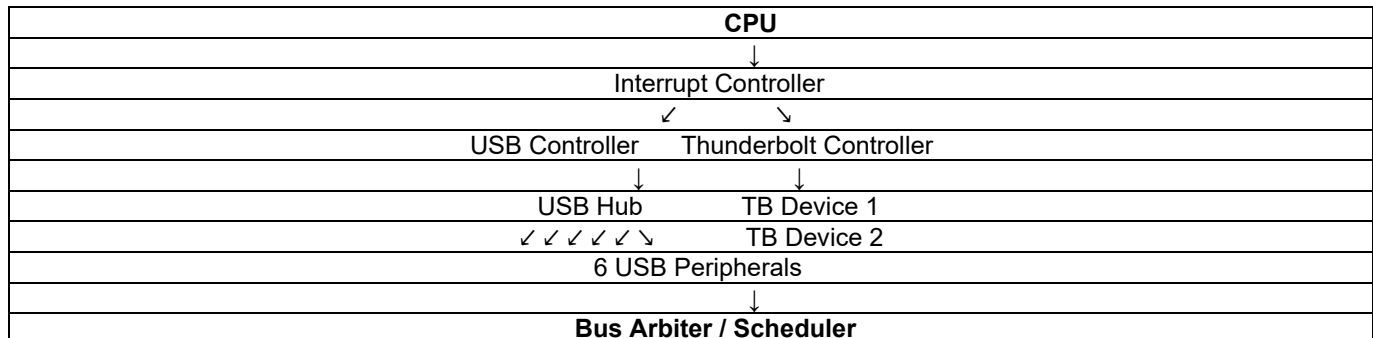
- Without DMA, the CPU would repeatedly copy large amounts of storage data, increasing CPU utilization.
- With DMA, bulk data is transferred directly between the NVMe controller and RAM.
- Interrupts are used for completion notification rather than for every byte or small transfer.

Result: NVMe over PCIe with DMA and completion interrupts provides high throughput while keeping bulk-data processing away from the CPU.

Question 3 – Multi-Device USB and Thunderbolt Communication Architecture

Given constraints: 6 USB peripherals, 2 high-speed Thunderbolt devices, no interrupt starvation, hardware and software interrupts, and fair bus arbitration.

Proposed Architecture



Design

- Connect the six USB peripherals through a USB controller/hub. Connect the two high-speed devices through the Thunderbolt controller.
- Use hardware interrupts for urgent device events such as completed transfers, errors, or device-status changes.
- Use software interrupts or deferred software handling for non-critical/background processing so hardware ISRs remain short.
- Use interrupt priorities but prevent low-priority devices from being blocked indefinitely. A bounded-priority or aging mechanism can raise the effective priority of a waiting device.
- Use round-robin or weighted round-robin bus arbitration for devices of comparable priority. This gives every requesting device a fair opportunity to use the shared communication resources.
- For high-priority traffic, allow priority arbitration, but impose a maximum service interval for lower-priority devices to prevent starvation.
- Keep hardware ISRs short and move lengthy protocol processing to deferred software tasks.

Interrupt-Starvation Prevention

Fairness rule: A device that has waited beyond a defined threshold receives increased scheduling priority. After service, its priority is restored. This combines responsiveness for urgent events with fairness for long-waiting devices.

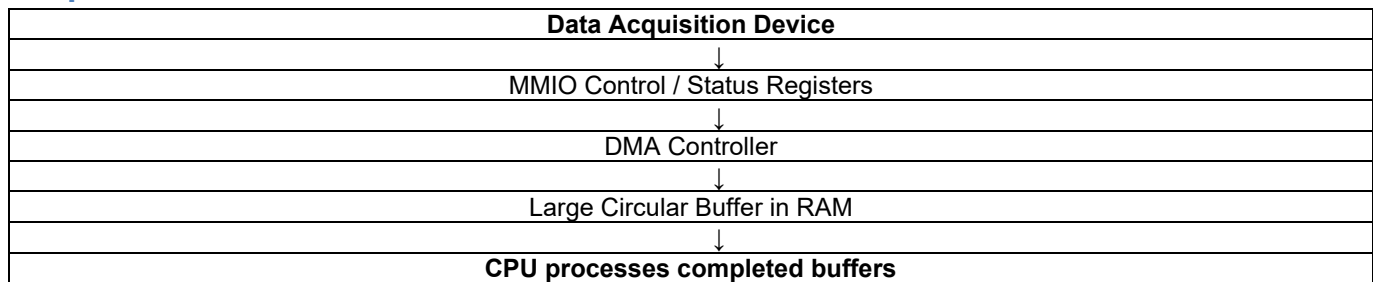
Result

The architecture supports all 8 peripherals, uses hardware interrupts for immediate events, software interrupts for deferred work, and fair arbitration to prevent any device from being permanently denied bus access.

Question 4 – 500 MB/s Data Acquisition System

Given constraints: continuous data stream of 500 MB/s, minimal CPU intervention, comparison of DMA and interrupt-driven I/O, and MMIO for device registers.

Proposed Architecture



DMA vs Interrupt-Driven I/O

Feature	DMA Transfer	Interrupt-Driven I/O
CPU involvement	Very low	High
Suitable for 500 MB/s stream	Yes	Poor / inefficient
Data movement	Device ↔ RAM directly	CPU handles each transfer
Interrupt frequency	Low; typically per buffer/block	Potentially very high
CPU utilization	Low	High
Best use	Large continuous data	Small or occasional transfers

Transfer-Rate Analysis

Data rate = 500 MB/s

For illustration, if the system generated one interrupt for every 4 KiB of data, the interrupt rate would be approximately:

$500,000,000 / 4096 \approx 122,070$ interrupts/second

This would create substantial CPU overhead if every interrupt required significant processing. With DMA, a much larger buffer can be transferred and only one interrupt can be generated per completed buffer, greatly reducing interrupt frequency.

MMIO Register Use

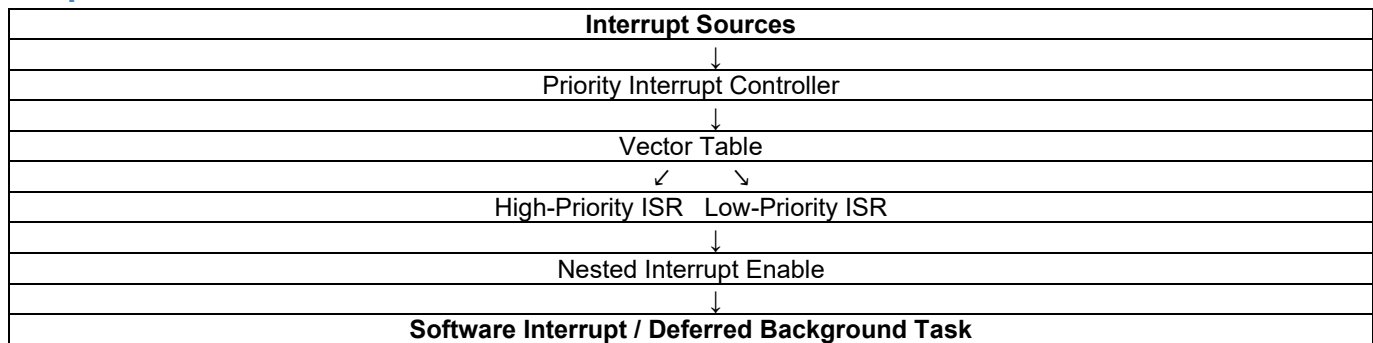
- Map the acquisition device's control, status, DMA address, length, and configuration registers into the CPU address space.
- The CPU configures the DMA transfer through these MMIO registers.
- After configuration, DMA continuously moves samples into a circular or double buffer in RAM.
- The CPU is interrupted only when a buffer reaches a defined completion point or when an error occurs.

Result: DMA is clearly preferable for the 500 MB/s continuous stream because it minimizes CPU intervention and supports sustained high-throughput data movement.

Question 5 – Nested Interrupt Handling Mechanism

Given constraints: nested interrupts, maximum latency < 10 μ s, vectored interrupts for high-priority devices, and software interrupts for background tasks.

Proposed Architecture



Mechanism

1. Configure the interrupt controller with priority levels. High-priority devices such as emergency sensors or real-time communication interfaces receive the highest priority.
2. Use a vector table so each high-priority interrupt directly selects its ISR, minimizing source-identification time.
3. When an interrupt arrives, save the minimum required CPU context and immediately branch to the corresponding ISR.
4. Allow a higher-priority interrupt to pre-empt a lower-priority ISR. This provides nested interrupt handling.
5. The ISR should acknowledge the hardware, capture essential data, and perform only time-critical work.
6. For lengthy or background processing, trigger a software interrupt/deferred task instead of performing the complete operation inside the hardware ISR.
7. After the higher-priority ISR finishes, restore the interrupted context and resume the lower-priority ISR.

Latency Requirement

Required maximum interrupt latency < 10 μ s

To meet this limit, the system must reserve sufficient execution time for interrupt entry, context saving, priority determination, vector lookup, and the start of the high-priority ISR. The high-priority ISR must also be short. A practical design target is to keep the complete interrupt-entry path comfortably below 10 μ s rather than operating exactly at the limit.

Example Priority Scheme

Priority	Interrupt Type	Handling
Highest	Critical real-time device	Vectored hardware ISR; may pre-empt lower priority ISR
High	Normal high-speed device	Vectored hardware ISR
Medium	Normal device	Hardware ISR with short processing
Low	Background processing	Software interrupt / deferred task

Result: Nested, prioritized vectored interrupts provide fast response for critical devices, while software interrupts keep background work away from the time-critical hardware ISR path.